Lab 2: A Control-Word Microprocessor

Microprocessors (010153521) | Semester 1/2026

Duration: 3 hours (in-lab) | **Topic:** ROM-as-control-store microarchitecture: no instruction register, no opcode decode – the ROM word *is* the control word

Objectives

- Explain the difference between a *hardwired FSM* control unit (Lab 1's EC-1, which decodes an opcode into signals) and a *control-word* machine (this lab, where every ROM bit *is* a control signal directly – there is no opcode, no Instruction Register, and no decode step at all).
- Build a self-addressing program counter: a register that feeds its own next value back from either an incrementer or a ROM-supplied jump address.
- Design the small piece of glue logic that sits *between* the ROM and the PC and decides, every cycle, whether to jump.
- Hand-assemble a control-word microprogram, load it into ROM, and verify a 0-to-10 counting sequence cycle by cycle.

Quick Reference: The Control Word

Program memory is 16 × 8-bit ROM. Unlike EC-1, there is **no opcode and no operand split** – every one of the 8 bits drives something directly:

bit 7	...	bit 4	bit 3	...	bit 0
`SEQ`	`LD_A`	`CLR_A`	`OUT`	`ADDR[3:0]`	

`LD_A`, `CLR_A`, `OUT` drive the Accumulator side directly. `ADDR[3:0]` is a jump target – meaningful only when `SEQ=0`. `SEQ=1` means "ignore `ADDR`, just advance to `PC+1`." There is one piece of logic *outside* the ROM, sitting between it and the PC:

$$\texttt{PC\_load} = \overline{\texttt{SEQ}} \cdot (\texttt{OUT} + \texttt{NEQ\_10})$$

where `NEQ_10` is a comparator flag (= 1 when Accumulator $A \neq 10$). When `PC_load=1`, `PC` ← `ADDR`; otherwise `PC` ← `PC+1`. This one equation is the *entire* control unit – there is nothing else.

Datapath

Figure 1 is the PC side: a self-addressing counter that can be redirected by the ROM's own `ADDR` field. Figure 2 is the Accumulator side.

The Microprogram: Count 0 to 10

Addr	Meaning	SEQ	LD_A	CLR_A	OUT	ADDR	Hex
0	Clear A	1	0	1	0	0000	0xA0
1	Output A	1	0	0	1	0000	0x90
2	Increment A	1	1	0	0	0000	0xC0
3	Wait (no-op)	1	0	0	0	0000	0x80
4	Jump to 1 if $A \neq 10$	0	0	0	0	0001	0x01
5	Jump to self, keep outputting	0	0	0	1	0101	0x15

Before you build anything: using the `PC_load` equation from the summary box, trace what happens at address 4 (`SEQ=0`) when $A \neq 10$, and separately when $A = 10$. Then trace address 5: why, once reached, does the machine loop on itself *forever* regardless of `NEQ_10`?

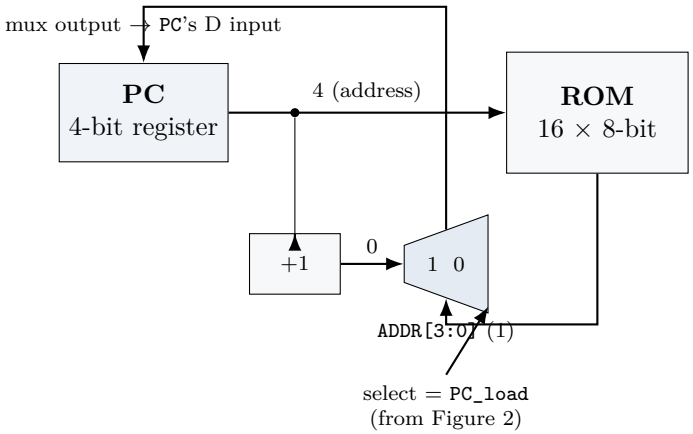

Figure 1: The PC side. PC addresses ROM directly – there is still no MAR. Every clock edge, PC latches *either* PC+1 (mux input 0, the normal case) *or* ROM's own ADDR[3:0] field (mux input 1, a jump), chosen by PC_load.

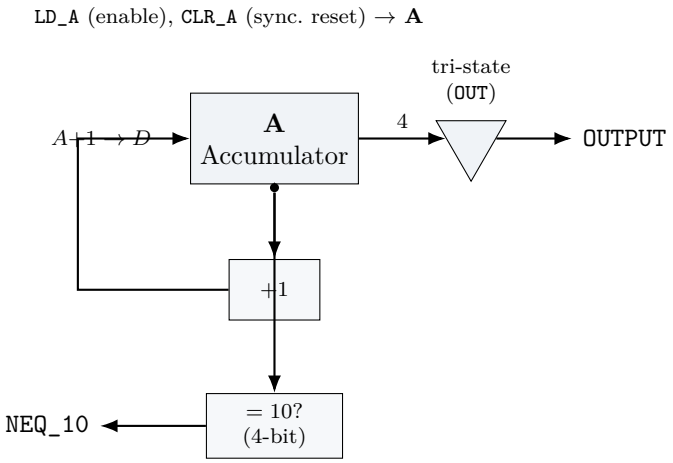

Figure 2: The Accumulator side. A either holds, clears (CLR_A), or loads $A+1$ (LD_A); a tri-state buffer drives A onto OUTPUT when OUT=1; a 4-bit comparator produces NEQ_10 whenever $A \neq 10_{10} = 1010_2$.

Quick Experiments (Try It)

Try It 1: PC jump mux, in isolation

Build just the PC side of Figure 1 (register, incrementer, mux – feed `ADDR` and `PC_load` from switches, not yet from ROM).

1. With `PC_load=0`, does `PC` count up on every clock edge?
2. Poke `ADDR=0101` and set `PC_load=1`. What is `PC` after the next clock edge? What if you leave `PC_load=1` and keep clocking?

Try It 2: Accumulator + comparator, in isolation

Build just Figure 2 (feed `LD_A`, `CLR_A`, `OUT` from switches).

1. Clear A, then clock `LD_A=1` ten times. Does `NEQ_10` go low exactly when A reaches `1010`?
2. With `OUT=0`, does the `OUTPUT` bus show anything? What does Logisim display for a tri-state buffer's output when disabled?

Try It 3: The glue logic alone

Build $\texttt{PC\_load} = \overline{\texttt{SEQ}} \cdot (\texttt{OUT} + \texttt{NEQ\_10})$ from a NOT gate, two AND gates and an OR gate, driven by three switches (`SEQ`, `OUT`, `NEQ_10`).

1. Fill in all 8 rows of its truth table by hand, then check every row against your circuit.
2. Which single input, if you don't know the other two, tells you the most about whether `PC_load` will be 1?

In-Lab Exercises (target: 3 hours)

In-Lab Exercise 1: PC side *(25 min)*

Build Figure 1 in full: the 4-bit `PC` register (latching every clock edge, no separate enable), the `+1` incrementer, and the `MUX`. Wire `PC`'s output directly to where `ROM`'s address will go.

In-Lab Exercise 2: Accumulator side *(30 min)*

Build Figure 2: the 4-bit Accumulator (synchronous `CLR_A` and `LD_A`), a `+1` block feeding its D input, a tri-state output buffer, and a 4-bit equality comparator producing `NEQ_10`.

In-Lab Exercise 3: ROM and the control word *(20 min)*

Add a 16 × 8-bit **ROM**, addressed by `PC`. Split its 8-bit output into `SEQ`, `LD_A`, `CLR_A`, `OUT` (1 bit each) and `ADDR[3:0]` using a Splitter. Wire the three Accumulator control bits straight to Exercise 2's circuit, and `ADDR` straight to Exercise 1's mux input `1`.

In-Lab Exercise 4: The glue logic *(15 min)*

Build $\texttt{PC\_load} = \overline{\texttt{SEQ}} \cdot (\texttt{OUT} + \texttt{NEQ\_10})$ from Try It 3, and wire it to the PC's `MUX` select input. This is the *entire* control unit for this machine – notice there is no state register anywhere in it.

In-Lab Exercise 5: Assemble, load, and run *(30 min)*

Wire Exercises 1–4 together. Hand-assemble the microprogram from the table above and load it with ROM's *Load Image*:

```
v2.0 raw
a0 90 c0 80 01 15 00 00 00 00 00 00 00 00 00 00
```

Single-step the clock and confirm: `OUTPUT` shows `0, 1, 2, ..., 10` in turn, one value per pass through address 1, and then the machine settles at address 5, continuously re-displaying `10` forever. ☐ Output sequence matches. ☐ Machine settles at address 5 (watch `PC` stop advancing past 5).

AI Collaboration

Try these prompts with an AI tool:

- "What is the architectural difference between a machine where the ROM word is decoded by an FSM, versus one where the ROM word directly drives the datapath?"
- "Given `PC_load = NOT(SEQ) AND (OUT OR NEQ_10)`, is there a smaller circuit that computes the same function?"
- "What Logisim-Evolution component implements a 4-bit magnitude comparator, and how do I wire one input to a fixed constant?"

Critical check – verify, don't just accept: Ask the AI whether this machine's control unit qualifies as a "Moore machine," a "Mealy machine," both, or neither. Then justify the answer yourself, in writing, from the actual circuit – `PC_load` depends combinationally on `NEQ_10`, which itself depends on the *current* value of a register (`A`), not on any external input. Does that change the classification the AI gave you?

Know the limit: AI tools often default to describing microprocessors in terms of fetch/decode/execute because that is the overwhelmingly common case in their training data. This machine has no decode stage at all – if you ask a leading question ("how does this machine decode its opcode?"), many models will happily invent a decode step that doesn't exist here rather than telling you the premise is wrong.

Take-Home (Paper Work). Solve the following on paper without using a computer or compiler. Hand-write your answers; submit at the start of the next lab. Goal: prepare you to dry-code during the written exam.

Trace & Predict 1: Six clock cycles from reset

Assume `PC` resets to 0. Fill in a table with columns *Cycle, PC, Control word (hex), A, OUTPUT* for the first six clock edges. At which cycle does `OUTPUT` first show a nonzero value?

Find the Bug 1: A swapped mux input

A student wired `ROM`'s `ADDR[3:0]` into the PC mux's input `0` and the `+1` incrementer into input `1` (i.e. the two mux inputs from Figure 1 are swapped), but the `PC_load` equation is otherwise correct and unchanged. Predict exactly what `PC` does starting from address 0, for at least the first 8 cycles. Does the machine still count correctly?

Write Code on Paper 1: Design a count-down microprogram

Using the same control-word format, write a new 16-word microprogram that counts *down* from 10 to 0 (instead of up), still outputting every value and settling into a self-loop at 0. You will need a different comparator target (`NEQ_0`, i.e. $A \neq 0$) and a decrementer instead of an incrementer. Write out the full table (address, mnemonic meaning, all five control fields, hex) as in the Quick Reference above.

Draw on Paper 1: Control-word datapath, from memory

Without looking at Figures 1–2, draw the complete datapath from memory: `PC`, the incrementer, the mux, `ROM`, the Splitter's five output fields, the Accumulator, its `+1` block, the tri-state buffer, the comparator, and the glue logic feeding the mux's select line. Compare against the real figures afterward and mark anything you got wrong or omitted.

Reflection

In one short paragraph, compare this lab's control unit to Lab 1's EC-1. Both machines execute a sequence of "instructions" stored in ROM and both can jump conditionally – but one has an Instruction Register, an opcode field, and a multi-state hardwired FSM, while the other has neither. What did EC-1's FSM *buy* you that this machine does not have? What did this machine give up to avoid needing one?